

Paxos 최종 발표

이준서



목차

- 합의 알고리즘의 목적
 - Paxos 합의 알고리즘 목표
- Single-paxos 알고리즘
 - 알고리즘 동작 과정
 - 한계점
 - Liveness 보장 알고리즘
- Multi-paxos 알고리즘
 - 알고리즘 동작과정
 - 세부 이슈

합의 알고리즘의 목적



분산 시스템에서 합의 알고리즘 목적

- 분산되어 있는 모든 참여자가 동일한 상태 또는 결정을 일관되게 도출할 수 있도록 보장

상태기계에서의 합의

- 상태기계 정의
 - 입력을 필요로 하고 입력으로부터 출력이 생성되며 기계 자체의 상태 정보 저장
- 상태기계에서 합의의 목적
 - 모든 상태기계들이 같은 상태를 가지도록 함
 - 분산화 되어있는 각 상태기계들이 동일한 내용의 작업을 동일한 순서로 실행

상태기계에서 Paxos 합의 알고리즘 목표

- 일관된 상태 전환 유지
 - 분산화 되어있는 각 상태기계들이 동일한 내용의 작업을 같은 순서로 실행하게 함
- 장애 허용성
 - 일부 노드가 실패하더라도 시스템이 계속해서 정상적으로 작동
 - 모순된 내용의 작업을 수행하지 않음
- 진행성 보장
 - 시스템이 교착 상태에 빠지지 않고 계속해서 상태 전환을 진행할 수 있도록 보장

Paxos를 통한 합의 알고리즘 목적 달성

➤ Single-paxos 알고리즘

- 모든 노드가 동일한 하나의 명령을 수행하도록 함
- 일관된 상태 전환 유지 및 장애 허용성 보장

➤ Multi-paxos 알고리즘

- 모든 노드가 동일한 명령을 동일한 순서대로 처리함
 - 노드가 수행할 명령을 각각의 Single-paxos 실행단위로 처리

Single-paxos 알고리즘



Single-paxos의 목적

- 절반 미만의 참여자가 실패해도 모든 노드가 동일한 하나의 명령을 수행하도록 함
- 정족수의 정의
 - 분산시스템에서 합의 시 일관성 보장위해 전체 노드 집합 중 반드시 참여해야 하는 최소한의 노드 수
- Paxos는 f 개의 노드가 실패하더라도 최소 $2f+1$ 개의 노드를 통해 합의 보장
- 언제나 Safety 보장
 - Safety: 시스템이 항상 일관된 상태로 도달함
- 정족수가 과반수인 이유
 - 두 개의 정족수 집합 간에 언제나 겹치는 노드가 존재
 - 겹치는 노드가 있기 때문에 두 개의 정족수 집합은 모순된 결정을 할 수 없음

합의에 참여하는 구성요소

➤ Proposers

- Client에게 요청을 받고 특정 값이 선택 완료 되어질 수 있도록 제안함
 - 값: Client의 Command

➤ Acceptors

- 제안을 검토하고 수락할지 결정함

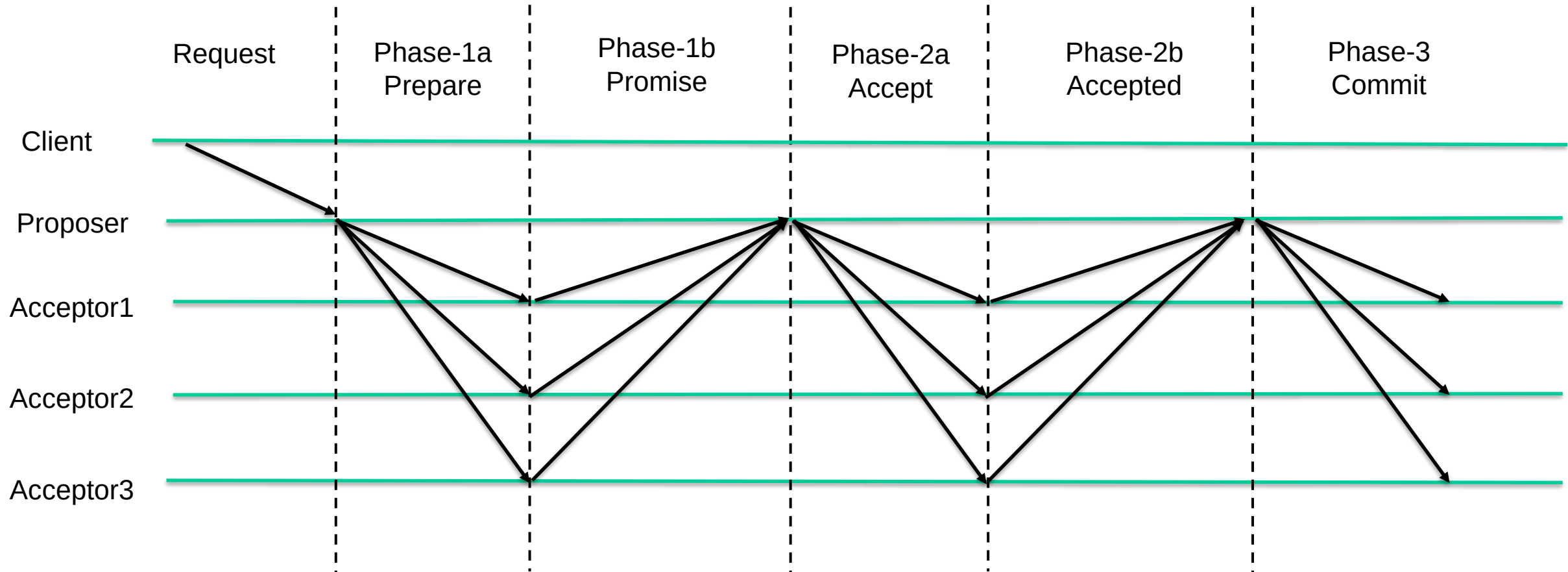
➤ Learners

- 최종적으로 합의된 값을 받아들이고 이를 다른 노드에 전달함
- 합의를 시작한 Proposer가 Distinguished Learner의 역할을 함
 - 합의를 시작한 Proposer만이 다른 노드에 합의된 값을 전달

Single-paxos 알고리즘 진행

➤ 2번의 메시지 교환으로 합의하고 Commit

- 값과 정족수 집합 결정하는 Phase-1, 결정된 값에 대해 투표가 이루어지는 Phase-2



Phase-1 Prepare 단계

- **Proposer는 지금까지 제안되었던 제안번호보다 큰 제안번호로 제안을 시작**
 - **각 Proposer는 Unique 제안번호 가지고 있음**
 - 제안 Round번호와 Server ID를 조합하여 Unique
- **Proposer가 모든 서버에게 Prepare(n) 메시지 Broadcast**
 - **n: 제안번호**

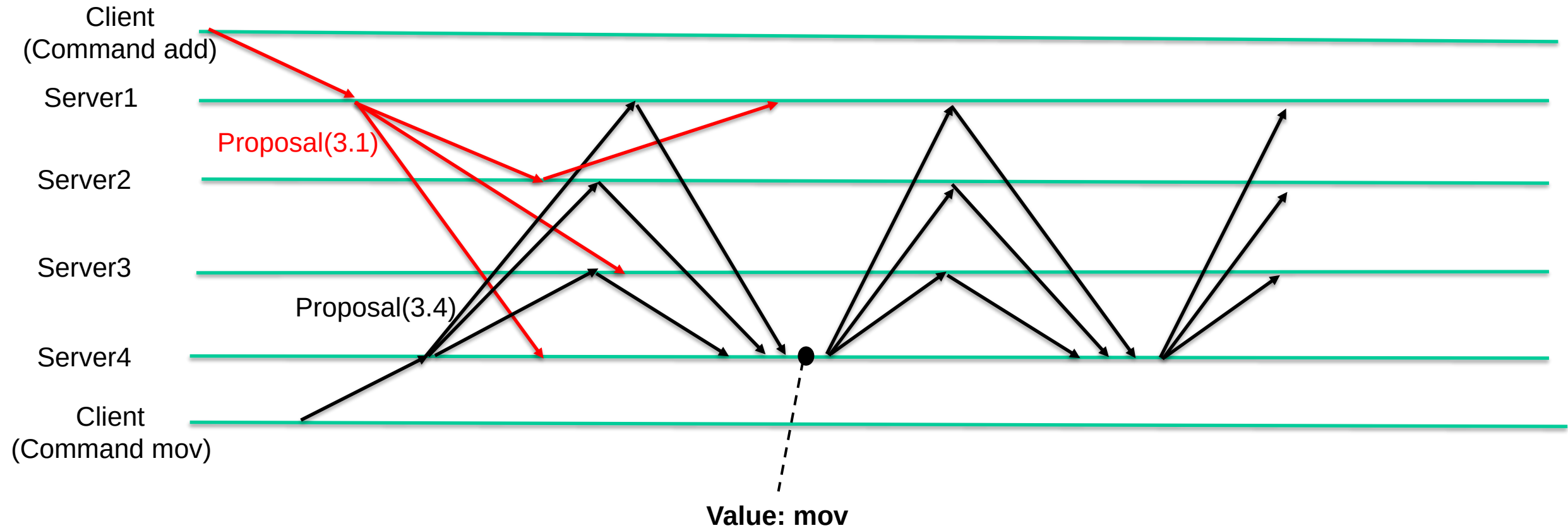
Promise 단계

- Acceptors가 Prepare(n)메시지를 받고 약속을 하는 단계
 - n보다 작은 제안에 응답하지 않을 것이라는 약속
- Acceptors는 가장 최근에 Promise한 제안과 Accepted한 제안을 기억함
 - Acceptors는 Promise한 제안보다 n이 더 작다면 무시
 - Acceptors가 Proposer에게 최근 Accepted한 제안의 번호와 값을 보냄
 - 가장 최근에 Accepted 제안: Accepted Proposal
 - 만약 Accept하지 않았다면 nil 보냄

Promise 단계의 예시

➤ Server1이 제안 3.1, Server4가 제안 3.4 를 시작

- Server2는 더 큰 번호의 제안 받아서 3.4에 응답, Server3은 Promise에 의해 3.1에 응답하지 않음
- Server4의 제안 값 mov 선택됨

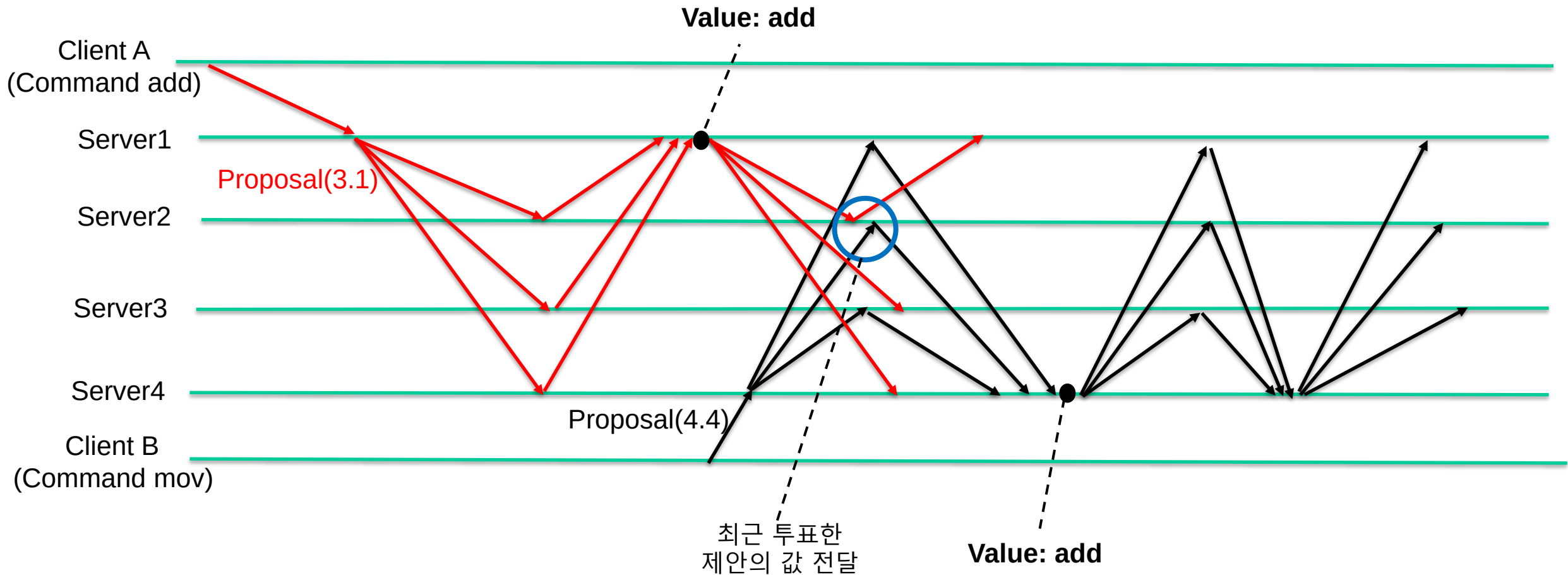


Phase-2 Accept 단계

- **Proposer가 만약 과반수의 Acceptors에게 응답을 받으면 정족수 집합과 제안의 값 결정**
 - 응답한 과반수의 Acceptors가 정족수 집합
 - Acceptor에게 응답 받은 Accepted Proposal 중 가장 높은 번호 제안의 값을 n번 제안의 값으로 결정
 - 만약 응답 받은 Accepted Proposal이 모두 nil 이라면 Client의 request를 값으로 결정
- **Proposer가 정족수 집합에게 Accept(n, v) Broadcast**
 - v: 제안의 값

Accept 단계에서 값 선택 예시

- Server1 제안의 값 결정된 이후 Server4 제안 시작
 - Server2가 제안을 Accept 했기 때문에 Server4 제안의 값은 Client A의 Command가 됨



Accepted 단계

- Acceptors가 다른 제안에 Promise하지 않았다면 $\text{Accept}(n, v)$ 의 제안번호 n 과 값 v 를 Accepted하고 응답

Commit 단계

- Proposer가 정족수 집합 모두에게 응답 받았다면 제안의 값이 선택됨
 - 선택된 값이 모든 상태기계가 수행해야 할 command
- Proposer는 모든 Acceptors를 대상으로 선택된 값을 전파

Single-paxos 알고리즘의 의의 및 한계점

➤ Single-paxos 알고리즘의 의의

- 어떠한 경우에도 언제나 Safety 보장
 - Proposer 여러 명이어도 Safety 보장
 - ✓ Phase 1 값 선택 방법에 의해서 보장됨

➤ Single-paxos 알고리즘의 한계점

- Liveness 보장되지 못함
 - FLP Impossibility에 따라서 한 노드의 실패에도 합의 도달 보장되지 않음
 - ✓ 노드의 실패와 메시지 지연 구분할 수 없어서 Proposer는 무제한 대기
 - ✓ Timeout 도입
 - Client의 요청이 제안되고 결국 끝이 난다는 점을 보장하지 못함
 - ✓ 점점 커지는 번호를 가진 제안이 다른 Proposer에 의해 자주 시작되면 서로의 진행을 막음
 - ✓ Leader 선출

Liveness 보장 Single-paxos

➤ Leader 선출

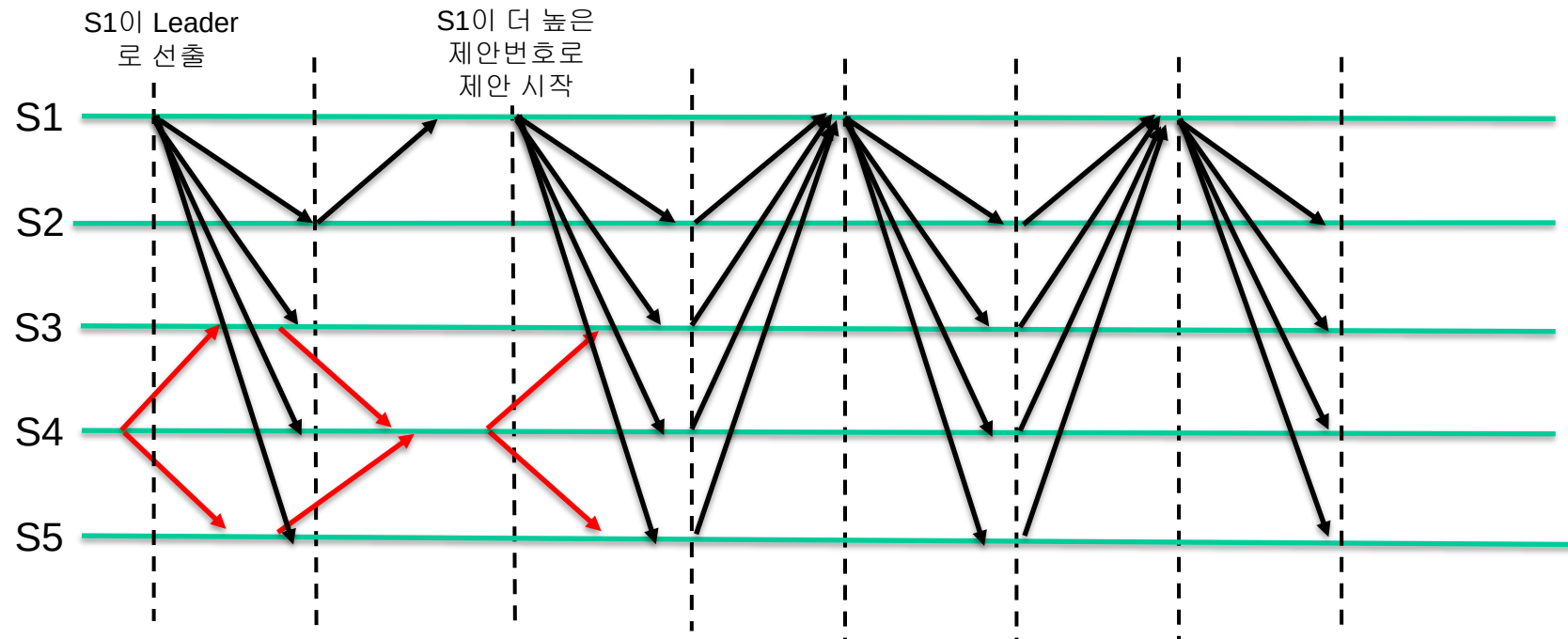
- 모든 클라이언트의 요청을 단 하나의 서버에게만 보냄
- 그 서버만이 Proposer의 역할을 함

➤ Timeout 설정

- Message 전달과 Server에서 Message 처리하는 시간의 Upper Bound를 둠
- 지연되거나 실패한 노드 감지하여 시스템이 무한 대기 상태에 빠지는 것 방지

Liveness 보장 방법

- 메시지 보내는 시간 3ms, 메시지 받고 처리하는 시간 5ms의 timeout
- 정상 Case에서는 Phase 1, 2 각각 16ms만큼, Phase 3 8ms로 40ms 걸림
 - Phase1, 2가 16ms 타임아웃까지 성공하지 못하면 실패로 간주하고 더 높은 제안번호로 다시 시작
 - 이전 Leader가 더 큰 제안번호로 제안했을 때도 56ms에 Commit 보장
 - 중간에 메시지 지연 및 실패 없다고 가정



Multi-paxos 알고리즘



연속적인 합의를 위한 Multi-paxos로의 확장

- Single-paxos를 확장하여 Sequential한 Command에 대한 연속적인 합의를 가능하게 함
 - 각각의 실행 시킬 Command가 독립적인 Single-paxos의 사례
 - Command의 일관성과 순서를 유지하는데 초점을 맞춤

Multi-paxos 추가된 개념 정의

- 로그
 - 분산 시스템에서 처리해야 할 명령들의 전체 기록
- 로그 Entry
 - 로그 내에서 각각의 개별 명령을 나타내는 단위 요소
- 로그 Index
 - 로그 내에서 각 로그 Entry의 위치를 나타내는 고유 번호

상태기계 복제와 로그 복제

➤ 상태기계 복제

- 여러 노드가 동일한 명령을 동일한 순서로 처리하여 모든 노드가 일관된 상태 유지
 - 상태 값을 복제하는 것이 아닌 동일한 명령을 동일한 순서로 처리하여 같은 상태를 유지하도록 함

➤ 로그 역할

- 상태 기계에 입력된 명령의 순서를 기록
- 트랜잭션의 순서와 내용을 로그에 기록하고 이를 복제하여 모든 노드가 일관된 트랜잭션 처리 보장

➤ 로그 복제

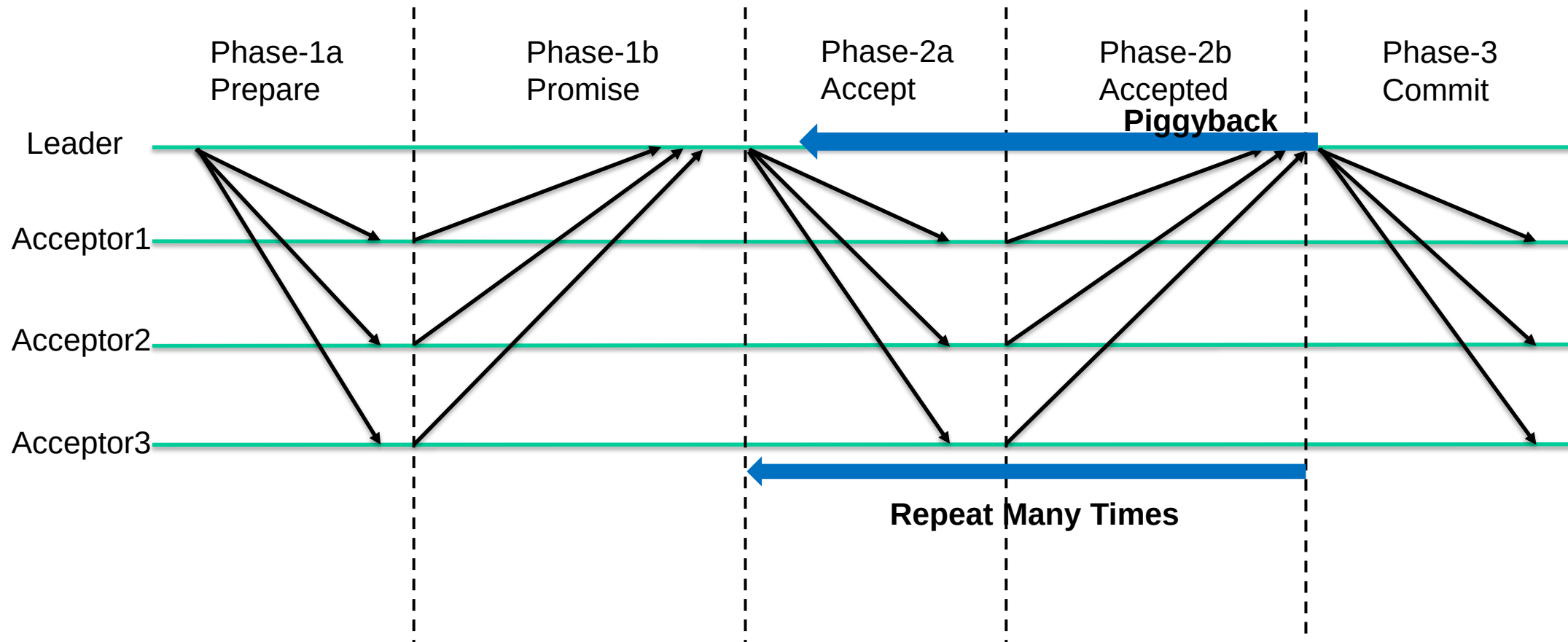
- Leader가 기록한 로그를 Acceptor들에게 복사하여 시스템 내 모든 노드가 동일한 명령을 처리
 - 데이터 일관성 및 장애 허용성의 목적

Multi-paxos 통한 합의 알고리즘의 목적 달성

- 분산화 되어있는 각 상태기계들이 동일한 내용의 작업을 같은 순서로 실행하게 함
 - 여러 명령이 정확한 순서로 복제되어 모든 노드가 동일한 로그 상태 유지하는 것 보장
 - 각 상태기계가 동일한 로그 내용과 순서를 실행하도록 보장
- Multi-paxos의 목적
 - 복제된 로그의 생성

Multi-paxos 알고리즘 진행

- Phase-1은 Leader 선출 때 한 번만 진행하고 Phase-2와 Phase-3만 반복
- Commit 메시지와 다음 로그 Entry에 대한 Accept 메시지가 같이 보냄



Multi-paxos Issues

- Phase 1 생략 최적화
- Client Request에 어떤 로그 Entry를 사용할지 결정 방법
 - Index 선택 방법
- Leader 선출 프로세스
- Full Replication

Phase 1 생략 최적화

➤ Phase 1 목적

- 이전 제안을 막음
- Leader가 가지고 있지 않는 정보 알아내기 위함
 - Leader는 합의하지 못했지만 이전 Leader에 의해서 Acceptor들은 합의 완료한 로그 Entry
 - Acceptor들이 이전에 Accepted한 제안의 제안번호, 값, 로그 Index

➤ Multi-paxos에서는 Phase 1에서 전체 로그 Entry에 대한 Prepare 한 번에 진행

➤ Phase 1을 통해 Leader가 가장 최근의 합의 결과를 가지고 있는 것 보장

- 모든 로그 Entry에 대한 투표 결과 알고 있음

➤ Leader 바뀔 때만 Phase 1 필요

	1	2	3	4	5	6	7
S1 (Leader)	shl	jmp	mov			ret	
S2	shl	jmp		add	sub	ret	
S3	shl	jmp	mov		sub	ret	

<로그 예시>

(색칠 부분은 합의 완료 된 Entry)

로그 Entry 선택 방법

- Phase 1 통해서 Leader가 가지고 있지 않은 합의 완료된 Entry는 없음 보장
- Leader의 로그 중 합의되지 않은 첫 번째 로그 Index 기준
 - 과반수가 같은 값에 투표한 로그 Entry는 Phase 2 통해 합의
 - 3번 로그 Entry의 경우 mov로 합의 진행
 - 과반수가 아닌 로그 Entry또한 Phase 2 통해 합의
 - Accept단계에 의해서 Leader는 이미 투표한 값을 가지고 합의 진행할 것임
- 새로운 Request가 들어왔을 때 아무도 투표하지 않은 로그 Entry에 대해서 합의 시작할 것
 - 그림에서 7번 로그 Entry

	1	2	3	4	5	6	7
S1 (Leader)	shl	jmp	mov			ret	
S2	shl	jmp		add		ret	
S3	shl	jmp	mov		sub	ret	

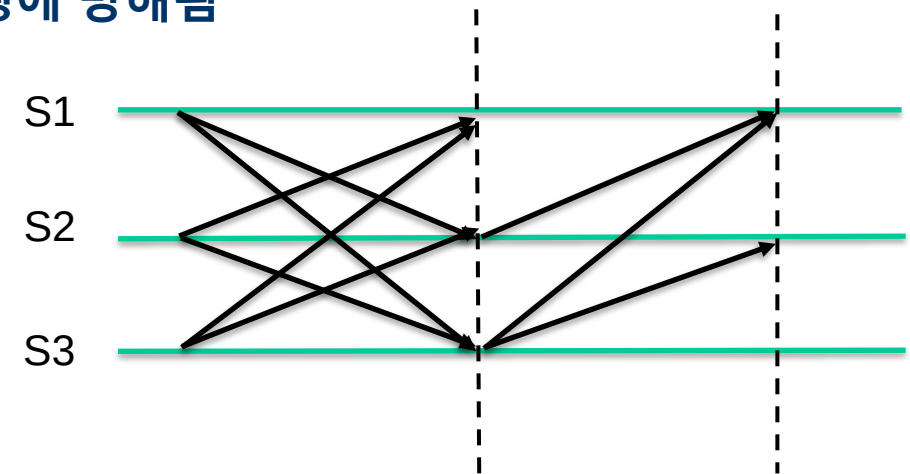
로그

Leader 선출 프로세스

- **Leader의 역할**
 - 로그 Entry를 제안하고 합의 과정을 주도
 - Leader만이 제안을 하고 Timeout을 통해 진행성 보장
- **Leader 선출 방법**
 - **Lamport 제안 방식**
 - [Part Time Parliament] 논문에서 제시
 - 가장 높은 서버ID 가진 서버가 Leader가 됨
 - **Google 제안 방식**
 - [Paxos Made Live] 논문에서 제시
 - 제안의 반복으로 안정적인 Leader가 지속적으로 제안

Lamport 제안 Leader 선출 방법

- 1. 주기적으로 각 서버는 자신의 ID가 담긴 메시지를 다른 모든 노드에게 보냄
- 2. 자신의 ID보다 낮은 메시지에 대해서만 응답
- 3. 일정 시간이 지나고 응답 받지 못한 서버가 자신을 Leader라고 생각
- Leader가 실패하면 Leader 선출 때까지 기다리고 새로운 Leader 선출 프로토콜 시작
- 한계점
 - 안정적인 서버가 Leader가 될 것이라는 보장이 안됨
 - 만약 S3의 Latency 높거나 자주 실패하는 서버라면 진행에 방해됨



<S1이 Leader되는 과정>

Google 제안 Leader 선출 방법

- Leader 선출을 따로 하지 않고 이전 Leader 실패 시 Leader 되고 싶은 노드가 제안 시작
 - Leader 실패했을 때 Client 요청을 먼저 받은 노드가 Leader 되고 싶은 노드
 - Phase 1 마친 Leader는 Lease를 주기적으로 갱신하여 다른 노드가 제안하지 못하도록 함
 - Lease: 일정 기간동안 독점적인 권한을 가지도록 허용하는 시간 제한 계약
- Leader 교체의 Overhead 줄일 수 있고 안정적인 Leader가 지속적으로 유지되도록 함

추가적인 리더 선출 방법

- Zookeeper에서 리더 선출
 - 각 서버가 자기보다 다음으로 높은 ID 가지는 서버 Monitoring
 - 다음 사람이 Leader이고 실패하면 Monitoring하고 있는 서버가 Leader가 됨
 - 아닌 경우에는 Timeout 기다리며 계속해서 Monitoring
 - Lamport 방식의 최적화 버전
- [Multi-Paxos: An Implementation and Evaluation]에서 리더 선출
 - 가장 최근에 제안을 성공시킨 노드를 Leader라고 간주
 - 주기적으로 모든 노드들은 Leader라고 간주하는 노드에게 메시지 보내서 Leader 누구인지 물어봄
 - 본인이 아닌 다른 노드가 Leader라고 하면 그 노드를 Leader로 간주
 - 일정시간 응답 없으면 실패했다고 판단하고 Client 요청 받은 노드가 본인을 Leader로 간주

Full Replication

➤ 문제 상황

- Leader가 아닌 다른 노드의 빈 로그 Entry에 대해서 채우는 것 보장하지 않음
 - 합의가 끝난 로그 Entry는 Leader가 더이상 제안 진행하지 않을 것임
 - 노드 장애 또는 네트워크 장애로 Learn하지 못한 로그 Entry 있음

➤ 로그 Replication 하는 경우

- Phase 3에서 Leader에게 합의된 값 Learn
 - 이미 합의 완료된 이전 로그는 Learn하지 못함
- Phase 1에서 Leader에게 빈 로그 Entry 값 물어볼 수 있음
 - Leader 교체할 때만 Phase 1 진행하기 때문에 효율적이지 못함
 - 추가적인 메시지 교환 필요할 수 있음

	1	2	3	4	5	6	7
S1 (Leader)	shl	jmp	mov	add	sub	ret	
S2	shl	jmp		add		ret	
S3	shl	jmp	mov		sub	ret	

로그

Full Replication Optimization

- Leader는 Phase 3 Commit 메시지를 보내는 대신 합의 완료된 Entry에 계속해서 Accept 메시지 보냄
 - Leader는 FirstUnchosenIndex 정보도 같이 보냄
 - 몇 번 Index까지 선택완료 되었는지 알 수 있음
 - Acceptor는 몇 번까지 선택되었는지 알 수 있고 Commit 가능
- Acceptor는 Phase 2-b Accepted 메시지에 Acceptor의 FirstUnchosenIndex 같이 보냄
 - Leader의 FirstUnchosenIndex보다 낮으면 그 Entry에 대해서 Leader가 Phase 3 Commit 메시지 보내줌

Raft와의 차이

- 최적화된 Multi-paxos는 Raft와 동작방식이 비슷함
 - Leader 선출하고 투표하여 Commit
- 하지만 동작 결과는 다름
 - Raft는 이전의 투표 값을 지지해주지 않음
 - Raft는 이전 Leader에 의해서 복제된 로그를 현재 Leader의 로그로 덮어쓰움
- Raft에서는 (e), (f) 케이스에 대해서 새로운 Restriction을 두고 제어함
 - Paxos는 (e), (f) 케이스가 생길 수가 없음
 - 이 전에 투표한 값들을 지지해주기 때문

